

NutriChain

I. Sommaire

NutriChain.....	1
I. Sommaire.....	2
II. Contexte du projet.....	3
III. Objectifs S.M.A.R.T.....	5
A. Objectif 1 - Traçabilité complète et standardisée des lots (cible : 20/06/2026).....	5
B. Objectif 2 – Alerte froide émise en moins de 30 secondes (p95) (cible : 15/03/2026).....	5
C. Objectif 3 – Sécurisation avancée des accès et de l'identité (cible : 01/03/2026).....	5
D. Objectif 4 – Expérience mobile fluide, rapide et résiliente (cible : 10/06/2026).....	6
E. Objectif 5 – Processus de rappel produit en moins de 15 minutes (cible : 22/06/2026).	6
F. Objectif 6 – Intégration normalisée avec les systèmes ERP/WMS/TMS (cible : 30/09/2026).....	6
G. Objectif 7 – Auditabilité renforcée et conformité réglementaire totale (cible : 30/09/2026).....	6
H. Objectif 8 – Mise en place d'une chaîne CI/CD fiable et industrielle (cible : 10/03/2026).....	7
IV. Parties prenantes.....	7
V. Rôles et responsabilités.....	8
VI. Diagramme de Gantt (M1 à M12).....	10
VII. Indicateurs KPI.....	11
VIII. Comparaison d'architectures candidates.....	14
A. Architecture monolithique 3-tiers.....	14
B. Architecture monolithique modulaire (Modulith) avec architecture hexagonale.....	15
C. Architecture microservices synchrones.....	16
IX. Audit d'outils nécessaires face aux besoins du projet.....	17
X. Benchmark des langages de développement.....	20
XI. Architecture retenue.....	22
XII. Langages de développement et outils retenus.....	24
A. Choix du backend et de l'API : Node.js.....	24
B. Choix du frontend web : Svelte.....	25
C. Choix du développement mobile : React Native.....	26
D. Choix de la base de données principale : PostgreSQL.....	27
E. Choix de l'ORM : Prisma.....	28
F. Usage complémentaire d'une base NoSQL : MongoDB (logging).....	28
G. Conclusion.....	28

II. Contexte du projet

Le secteur agroalimentaire est soumis à des exigences croissantes en matière de traçabilité, de sécurité sanitaire et de réactivité face aux incidents logistiques ou qualité. Les industriels, transformateurs, logisticiens et distributeurs doivent être capables d'identifier précisément l'origine des produits, de suivre chaque lot tout au long de son cycle de vie et d'intervenir rapidement en cas de non-conformité ou d'anomalie de température. Dans la réalité du terrain, ces entreprises fonctionnent encore avec une grande hétérogénéité d'outils : ERP incompatibles, tableurs utilisés comme système principal de suivi des lots, étiquetage non standardisé, capteurs de température déconnectés du système informatique et rappels produits gérés par e-mail.

Ce fonctionnement fragmente la donnée, complique les croisements d'informations et ralentit considérablement les processus critiques, en particulier les rappels produits et la surveillance de la chaîne du froid.

NutriChain, créé en 2021, intervient précisément dans ce contexte. L'entreprise fournit des solutions permettant de tracer les produits « de la ferme au rayon », de centraliser les événements logistiques selon les standards GS1/EPCIS et de surveiller la chaîne du froid en temps réel grâce à des capteurs IoT. Avec près de 18 millions de lots suivis, plus de 11 000 capteurs de température déployés et un p95 ingest→alerte actuellement à 35 secondes, NutriChain fait face à une demande croissante de la part de plus de 75 clients répartis en France, Belgique et Espagne.

L'ambition stratégique est d'accroître la fiabilité, la rapidité et l'automatisation des opérations, tout en préparant une expansion vers l'Allemagne et l'Italie à l'horizon 2026.

Les systèmes actuels des clients présentent plusieurs limites structurelles. Les processus de réception, transformation et expédition ne sont pas uniformisés, et les informations critiques (GTIN, lots, DLC/DLUO, SSCC) sont souvent saisies manuellement, sans standardisation fiable. La chaîne du froid repose encore sur des enregistreurs USB ou des capteurs non intégrés, ce qui empêche l'obtention d'alertes en temps réel et génère un risque sanitaire réel.

En cas de rappel produit, l'absence de traçabilité consolidée nécessite des opérations manuelles longues, sources d'erreurs, souvent incomplètes et parfois trop tardives. Ces

contraintes augmentent significativement les risques opérationnels : rappel tardif, produits non conformes en rayon, données inexactes, interventions non documentées ou non auditées.

Le projet NutriChain vise à refondre et moderniser l'application existante pour fournir une plateforme unifiée, fiable et interopérable, capable de gérer la traçabilité complète des lots, la surveillance continue de la chaîne du froid et l'exécution de rappels produits rapides et ciblés. Cette refonte s'appuie sur des exigences fortes : disponibilité mensuelle supérieure à 99,9 %, p95 ingest→alerte froid inférieur à 30 secondes, stabilité mobile supérieure à 99,5 % et latence de scan inférieure à 500 millisecondes.

La plateforme doit également garantir une sécurité avancée fondée sur OIDC, MFA et un contrôle d'accès ABAC, tout en assurant une traçabilité rigoureuse des actions via des logs immuables (WORM) et un plan de reprise d'activité robuste (RTO 60–120 min, RPO 15 min).

Les enjeux internes sont tout aussi déterminants. Les parties prenantes (équipes qualité, production, logistique, DSI, magasins et consommateurs) attendent un système simple, rapide et fiable permettant d'améliorer le suivi opérationnel, d'automatiser les contrôles et de centraliser les données critiques. Les opérations doivent s'inscrire dans un cadre réglementaire strict, incluant HACCP, ISO 22000 et les normes GS1.

NutriChain doit également composer avec ses propres contraintes techniques : connecteurs ERP/WMS/TMS hétérogènes, stockage distribué, sécurité partielle, logs non WORM et observabilité encore insuffisante côté RUM.

Dans ce contexte, la refonte NutriChain n'est pas seulement une évolution technique : c'est une transformation profonde des processus métier autour d'une architecture standardisée, orientée microservices, pilotée par les événements EPCIS et capable de gérer efficacement les scénarios critiques tels que l'excursion de température ou le rappel produit. Le MVP visé doit démontrer la capacité à scanner un lot, réaliser une transformation simple, l'expédier et retrouver l'intégralité de sa traçabilité.

Cette base solide permettra d'étendre progressivement la couverture fonctionnelle, de renforcer la sécurité, d'améliorer la performance globale et de garantir une interopérabilité complète avec les systèmes des clients.

III. Objectifs S.M.A.R.T.

Dans le cadre de la refonte de la plateforme NutriChain, il est essentiel de formaliser des objectifs SMART clairs et partagés afin de guider la conception, prioriser les livrables et mesurer les progrès. Ces objectifs sont conçus pour être spécifiques, mesurables, atteignables, réalistes et datés. Ils traduisent les priorités opérationnelles observées sur le terrain (traçabilité, sécurité sanitaire, réactivité en cas de rappel, performance mobile et interopérabilité) et servent de base au pilotage du projet pour les équipes produit, développement, qualité et exploitation.

A. Objectif 1 - Traçabilité complète et standardisée des lots (cible : 20/06/2026)

Mettre en place un système EPCIS conforme GS1 permettant d'enregistrer et de restituer l'intégralité des événements logistiques (réception, transformation, agrégation, expédition) afin d'assurer une traçabilité amont/aval complète pour 100 % des lots couverts par le périmètre MVP, démontrable à la livraison du premier jalon.

B. Objectif 2 – Alerte froide émise en moins de 30 secondes (p95) (cible : 15/03/2026)

Le système doit pouvoir détecter une excursion de température et générer une alerte en moins de 30 secondes (p95). Cela implique un pipeline de streaming optimisé, une ingestion fiable et des mécanismes d'alerting tolérants aux pannes. L'objectif vise à réduire les pertes, limiter les risques sanitaires et permettre une intervention immédiate des équipes sur le terrain.

C. Objectif 3 – Sécurisation avancée des accès et de l'identité (cible : 01/03/2026)

L'objectif consiste à généraliser l'authentification OIDC, à rendre la MFA obligatoire pour les rôles critiques et à établir une autorisation dynamique via un modèle ABAC. Ce renforcement des accès doit être pleinement opérationnel pour le livrable L2, afin de garantir

une protection solide contre les accès non autorisés et d'assurer la conformité aux standards internes de sécurité.

D. Objectif 4 – Expérience mobile fluide, rapide et résiliente (cible : 10/06/2026)

Le scan des lots doit être instantané, avec une latence inférieure à 500 ms et un taux de crash inférieur à 0,5 %. L'application mobile doit fonctionner hors-ligne, stocker localement les scans et synchroniser automatiquement les données. Cet objectif vise à garantir un usage fiable dans des environnements industriels souvent contraints (zones froides, entrepôts métalliques, faible réseau).

E. Objectif 5 – Processus de rappel produit en moins de 15 minutes (cible : 22/06/2026)

Le système doit être capable d'exécuter un rappel complet (identification des lots concernés, ciblage, envoi des notifications et suivi des retraits) en moins de 15 minutes entre la décision initiale et la diffusion effective. Cet objectif est central pour réduire les risques sanitaires et améliorer la réactivité des équipes qualité.

F. Objectif 6 – Intégration normalisée avec les systèmes ERP/WMS/TMS (cible : 30/09/2026)

L'objectif est d'assurer une interopérabilité fluide avec les systèmes clients grâce à des connecteurs normés (SAP, Oracle, Navision, WMS, TMS) et un support EDI/CSV pour les environnements legacy. Deux connecteurs opérationnels doivent être livrés dès le second livrable, afin de réduire les ressaisies manuelles et renforcer la cohérence des données.

G. Objectif 7 – Auditabilité renforcée et conformité réglementaire totale (cible : 30/09/2026)

Le projet doit garantir une journalisation inviolable (WORM) couvrant l'ensemble des actions critiques, avec une rétention minimale de 30 jours. Les exigences HACCP, ISO 22000 et internes doivent être respectées grâce à une gestion stricte des logs, un RPO de 15 minutes et un RTO maximal de deux heures en cas d'incident majeur.

H. Objectif 8 – Mise en place d’une chaîne CI/CD fiable et industrielle (cible : 10/03/2026)

La CI/CD doit permettre un déploiement continu en environnement de test, intégrer des tests unitaires et d'intégration systématiques, et garantir un temps de build et de déploiement maîtrisé. L'objectif est d'éliminer les erreurs humaines, d'accélérer les mises en production et de fournir un pipeline stable dès le livrable L2, puis entièrement opérationnel pour L3.

IV. Parties prenantes

La refonte de la plateforme NutriChain implique un ensemble d'acteurs internes et externes dont les besoins, responsabilités et contraintes influencent directement la conception du système. Leur compréhension est indispensable pour aligner les objectifs du projet sur les réalités opérationnelles du terrain et garantir l'adoption de la solution. Les parties prenantes identifiées se répartissent en six catégories principales : Qualité, Production, Logistique, DSI, Magasins et Consommateurs (B2C).

Les équipes Qualité jouent un rôle central dans la traçabilité, le suivi des non-conformités et la gestion des rappels produits. Elles ont besoin d'un système fiable permettant d'identifier rapidement les lots concernés, de documenter les actions menées et de garantir la conformité aux normes HACCP et ISO 22000. Pour elles, la rapidité du rappel, la disponibilité des données et l'intégrité des logs WORM sont essentielles.

Les équipes de Production sont responsables de l'enregistrement des événements de transformation, d'agrégation et d'expédition. Leur efficacité dépend d'interfaces simples, rapides et adaptées aux contraintes d'un environnement industriel. Elles attendent un système permettant de réduire les saisies manuelles, d'éviter les erreurs de lot et de fluidifier la saisie et le scan grâce à des standards GS1/EPCIS.

Les équipes Logistiques gèrent les réceptions, les mouvements de stock, les palettes (SSCC) et les expéditions. Leur priorité est d'obtenir un suivi en temps réel des flux physiques et un accès immédiat aux événements EPCIS pour garantir la cohérence entre les stocks

théoriques et réels. Elles dépendent par ailleurs d'une application mobile robuste, fonctionnant hors-ligne et capable de synchroniser les scans dans des zones à faible couverture réseau.

La Direction des Systèmes d'Information (DSI) supervise l'intégration de la plateforme avec les systèmes internes et externes, notamment les ERP, WMS et TMS. Elle est particulièrement attentive aux aspects de sécurité (OIDC, MFA, ABAC), de disponibilité, de PRA/PCA et d'observabilité. La DSI attend une architecture standardisée, interopérable et évolutive, capable de s'intégrer dans un écosystème déjà complexe.

Les équipes Magasins interviennent principalement sur la consultation des lots, la vérification des dates, la gestion des retraits et l'exécution des rappels en rayon. Elles ont besoin d'un portail simple d'utilisation, rapide et accessible depuis mobile, permettant de confirmer la présence ou l'absence de produits concernés.

Enfin, **les Consommateurs (B2C)** doivent pouvoir vérifier un lot via un code ou un QR scan, consulter l'historique du produit et être informés efficacement en cas de rappel. Leur expérience doit être transparente, fiable et sécurisée, car elle influe directement sur la confiance dans la marque.

En résumé, ces différentes parties prenantes imposent une solution fiable, interopérable et adaptée aux réalités du terrain. Leur diversité souligne l'importance d'une plateforme capable de répondre à des exigences opérationnelles, réglementaires et techniques tout en garantissant une adoption fluide par l'ensemble de la chaîne de valeur.

V. Rôles et responsabilités

La réussite de la refonte de la plateforme NutriChain repose sur une coordination claire et une répartition précise des responsabilités au sein de l'équipe projet. Définir ces rôles permet de garantir la cohérence dans la réalisation des livrables, d'éviter les doublons et de responsabiliser chaque membre sur ses missions. L'équipe projet est composée de trois développeurs aux compétences complémentaires et d'un chef de projet assurant la supervision globale.

Nom	Rôle	Responsabilités
Paul	Chef de projet et développeur API	Coordination globale du projet, suivi du planning et communication avec le client. Développement de l'API principale. Implémentation des flux EPCIS, de la

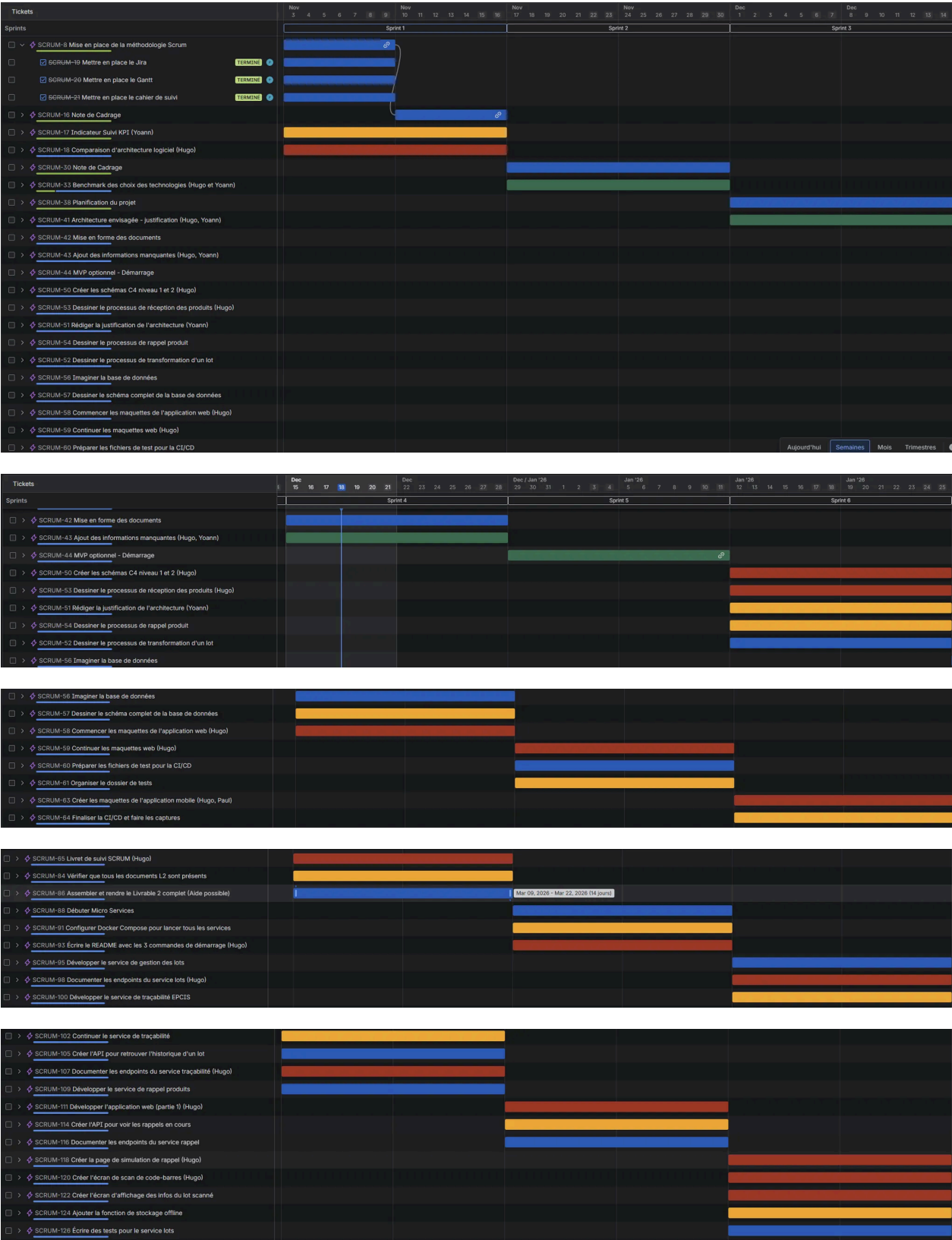
		traçabilité des lots et de la gestion des événements logistiques. Garantie de la qualité technique des livrables et du respect des exigences de sécurité et de performance.
Hugo	Développeur Front-end / Mobile	Développement des interfaces web et mobiles de la plateforme. Optimisation de l'expérience utilisateur pour des environnements industriels contraints (zones froides, faible connectivité). Conception et implémentation des écrans de consultation, de scan des lots et de suivi des alertes, en garantissant rapidité, fiabilité et ergonomie.
Yoann	Développeur polyvalent	Contribution transversale à l'ensemble du projet. Intervention selon les priorités sur l'API, le front-end ou l'intégration avec les systèmes clients (ERP/WMS/TMS). Support aux autres développeurs et adaptation rapide aux besoins du projet afin de garantir l'avancement des travaux dans les délais.

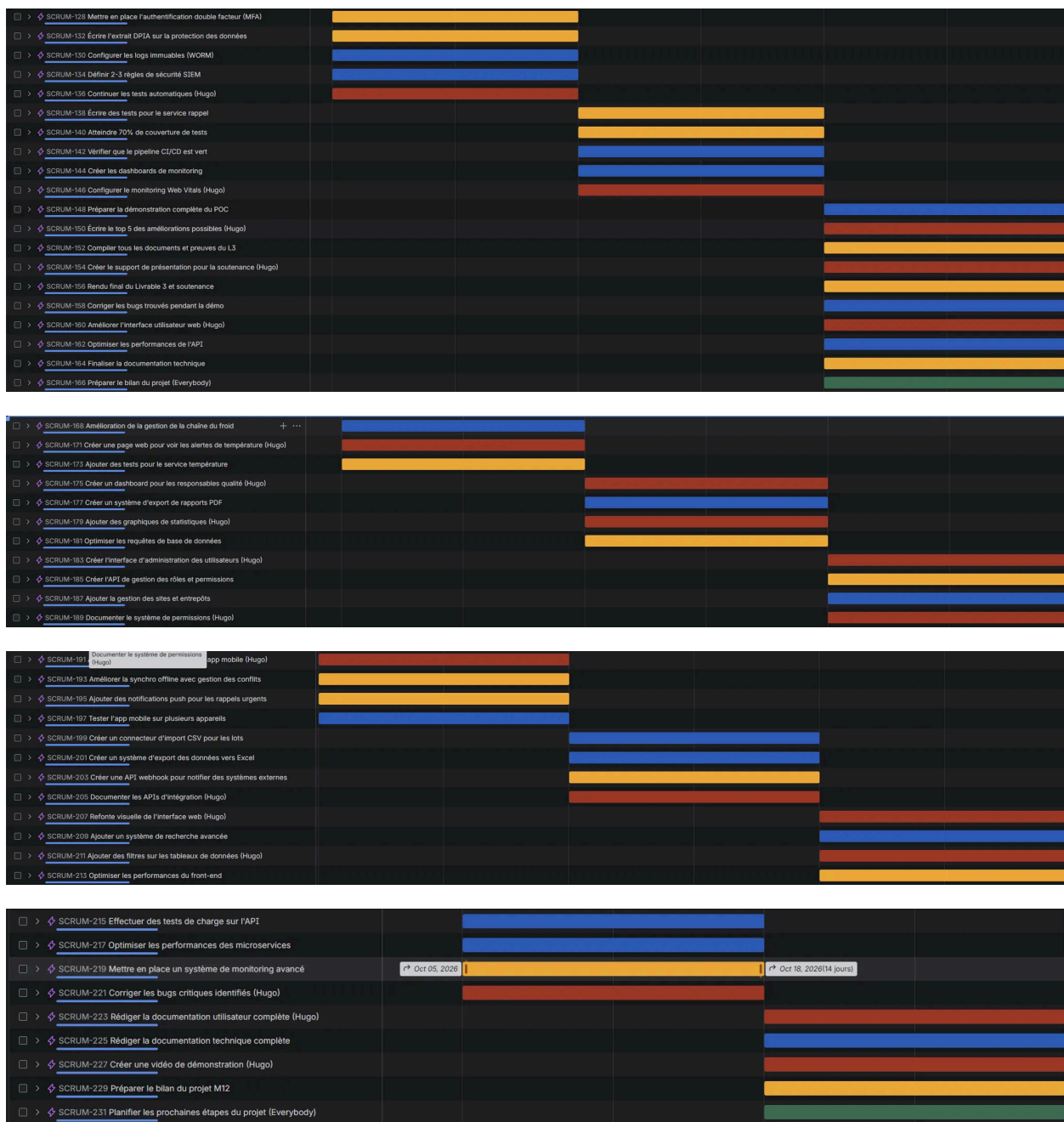
Cette répartition assure que chaque composant de la plateforme NutriChain est pris en charge par un responsable dédié, tout en maintenant une collaboration étroite entre les membres de l'équipe. L'organisation garantit une exécution efficace, conforme aux exigences du client, et facilite la maintenance et l'évolution future de la solution.

Le cadrage du projet NutriChain est désormais complet et partagé avec l'équipe. Nous validons le contexte métier, les objectifs SMART avec dates cibles, les parties prenantes et la répartition des rôles. Les responsabilités de chaque membre de l'équipe sont claires et alignées sur les besoins opérationnels. La méthodologie SCRUM, les KPI principaux et la planification préliminaire sont définis pour suivre l'avancement.

La prochaine étape du livrable consiste à passer à la conception détaillée : architecture cible, anticipation des besoins et préparation de l'infrastructure.

VI. Diagramme de Gantt (M1 à M12)





VII. Indicateurs KPI

KPI	Définition	Objectif / Seuil	Importance
Traçabilité (EPCIS)			
Couverture de traçabilité	Pourcentage de lots / palettes / produits complètement	> 98%	Indique si les données sont complètes et exploitables

	retracés (réception → transformation → expédition)		pour un rappel.
Taux d'évènements EPCIS valides	Pourcentage d'évènements conformes au standard GS1 (structure, champs obligatoires)	> 99%	Garantir l'interopérabilité avec ERP / WMS.
Temps moyen de reconstruction d'un arbre de traçabilité	Délai pour générer la vue complète d'un lot (upstream + downstream)	< 2 secondes	Impact direct sur la vitesse de décision lors d'un rappel
Taux d'erreurs de scan (mobile ou poste fixe)	Pourcentage de scans invalides, illisibles ou non reconnus	< 3%	Mesure la qualité des données entrantes et la facilité d'usage pour les opérateurs.
Chaîne du froid / IOT			
Temps p95 ingest → alerte	Temps entre lecture capteur et affichage alerte	< 30 secondes (exigence NFR)	KPI critique pour la sécurité alimentaire
Nombre d'excursions par mois	Nombre de dépassements de seuils détectés	< 15	Permet de repérer des sites problématiques
Durée moyenne d'excursion	Temps entre début d'excursion et action corrective	< 10 minutes	Impact sur le risque sanitaire et pertes produits.
Taux de perte de messages IOT	Pourcentage de mesures non reçues (packet loss)	< 0.1%	Indique la robustesse de l'ingestion IOT.
Rappels produits			
Temps médian entre décision et notification	Délai pour que les magasins/clients reçoivent la notification après décision de rappel	< 15 minutes	KPI stratégique pour la conformité réglementaire.
Taux de rappel complété	Pourcentage de quantités rappelées contre les quantités ciblées	> 90%	Mesure l'efficacité réelle du rappel.
Taux de faux positifs	Pourcentage de produits rappelés alors qu'il n'étaient pas concernés	< 1%	Impact direct sur coût et confiance clients.
Taux de clôture d'un rappel	Durée totale du début à la clôture	Selon les normes internes	Pour reporting interne (COFIL, QSE).
Performance et UX mobile			
Latence par scan	Temps entre scan et validation	< 500ms (NFR)	Impact sur ergonomie et productivité

Taux de crash mobile	Pourcentage de sessions se terminant par un crash	< 0.5% (NFR)	Garantit la fiabilité de l'application.
Temps de chargement des écrans critiques	Accès dashboard, scan, alerte	< 1 seconde	Impact adoption utilisateur
Interopérabilité / Intégrations			
Taux d'imports ERP/WMS réussis	Pourcentage de fichiers / API ingérées sans erreur	> 99%	Critique pour clients legacy.
Temps de traitement d'un import CSV/EDI	Parsing + validation + transformation	< 60 secondes	Pouvoir ne pas retarder la traçabilité
Taux de normalisation automatique	Pourcentage de données mal formées correctement corrigées	> 90%	Réduit le besoin d'intervention humaine
Sécurité et conformité			
Taux de tentatives d'accès non autorisées bloquées	Nombre d'incidents par mois	Tendance en baisse continue	Suivi cybersécurité
Utilisateurs MFA activité	Pourcentage de comptes sensibles	100% pour rôles importants / administrateurs	Garantir la conformité et l'auditabilité.
Logs disponibles et immuables	Pourcentage de logs conservés > 30 jours (WORM)	> 99.9% (NFR)	Exigence légale de traçabilité
SLA / Résilience			
Disponibilité du service	Pourcentage de l'uptime mensuel	> 99.9% (NFR)	Impact sur les opérations critiques
RTO	Temps de restauration après incident	60-120 minutes (NFR)	Limite interruption chaîne de traçabilité
RPO	Perte maximale de données	< 15 minutes (NFR)	Garantit l'intégrité des données IOT / EPCIS
Coûts et non qualité			
Coût mensuel des pertes produits	Valeur des produits détruits suite à excursion ou rappel	En baisse continue	KPI financier essentiel
Nombre d'incidents qualité	Rappels, excursions, erreurs de lot, etc.	En baisse continue	Permet classification des causes récurrentes
Adoption utilisateur			
Taux d'utilisation de l'app/scans	Nombre de scans par opérateur par jour	En croissance continue	Mesure adoption API & UI

Temps moyen de formation	Durée pour qu'un nouvel opérateur soit autonome	< 2 heures	KPI ergonomie et onboarding.
--------------------------	---	------------	------------------------------

VIII. Comparaison d'architectures candidates

Dans le cadre du projet NutriChain, une phase de réflexion préalable a été menée afin d'identifier les architectures logicielles les plus adaptées aux enjeux du système.

Cette analyse est réalisée indépendamment des choix technologiques afin d'évaluer les impacts de chaque architecture sur la maintenabilité, l'évolutivité, la disponibilité et la complexité globale du projet.

Trois architectures principales ont été étudiées :

- l'architecture monolithique 3-tiers,
- l'architecture monolithe modulaire associée à une architecture hexagonale,
- l'architecture microservices synchrones.

Les sections suivantes détaillent le principe de chacune de ces architectures, ainsi que leurs avantages et inconvénients dans le contexte spécifique du projet NutriChain.

A. Architecture monolithique 3-tiers

(API + logique métier + base de données unique)

L'architecture monolithique 3-tiers repose sur une application unique regroupant l'ensemble des fonctionnalités du système.

Cette application est structurée en trois couches principales :

- une couche de présentation ou d'API (exposition des endpoints),
- une couche de logique métier,
- une couche de persistance, reposant sur une base de données unique.

L'ensemble est déployé et exécuté comme un seul artefact logiciel.

L'un des principaux avantages de cette architecture est sa simplicité de mise en œuvre. Le déploiement est direct, la configuration est centralisée et les interactions entre composants sont réalisées par des appels directs en mémoire.

Cette simplicité facilite également le débogage, notamment en phase de démarrage du projet, car l'ensemble du système peut être analysé depuis un point unique.

Dans un contexte de développement initial ou de preuve de concept, cette approche permet d'obtenir rapidement une première version fonctionnelle du produit.

Cependant, cette architecture présente plusieurs limites pour un projet tel que NutriChain.

Le couplage fort entre les différentes parties du système implique que toute évolution fonctionnelle ou technique peut impacter l'ensemble de l'application. Cela rend le travail en parallèle plus complexe et augmente le risque de régressions.

Par ailleurs, NutriChain intègre des aspects hétérogènes, notamment la gestion d'événements de traçabilité, des alertes temps réel et des flux simulés liés à l'IoT. Dans une architecture monolithique 3-tiers, ces traitements peuvent entrer en concurrence pour les mêmes ressources (runtime, base de données), ce qui peut entraîner des problèmes de performance ou de saturation lorsque le volume d'événements augmente.

Enfin, cette architecture offre une faible flexibilité d'évolution, notamment si le système devait à terme être découpé ou distribué.

L'architecture monolithique 3-tiers constitue une solution simple et efficace pour des applications de petite taille, mais elle montre rapidement ses limites dès que les domaines métier se complexifient ou nécessitent une évolution indépendante.

B. Architecture monolithique modulaire (Modulith) avec architecture hexagonale

L'architecture monolithe modulaire repose sur une application unique, mais dont le code est structuré par domaines métier clairement identifiés (par exemple : catalogue produit, traçabilité, rappels, alertes).

Chaque module est conçu comme une unité cohérente, avec des frontières explicites.

L'utilisation de principes issus de l'architecture hexagonale renforce cette séparation en distinguant :

- le cœur métier (indépendant des technologies),
- les ports (interfaces définissant les besoins du métier),
- les adaptateurs (implémentations techniques telles que l'API REST ou l'accès à la base de données).

Cette architecture combine la simplicité de déploiement du monolithe avec une organisation interne beaucoup plus robuste.

La structuration par domaines réduit le couplage entre les fonctionnalités et améliore la lisibilité du code, ce qui est particulièrement bénéfique dans un projet collaboratif.

Les frontières explicites introduites par l'architecture hexagonale facilitent :

- l'écriture de tests unitaires et d'intégration,
- l'évolution du système sans impact global,
- le remplacement ou l'adaptation de composants techniques.

Dans le contexte de NutriChain, cette approche permet de modéliser correctement les domaines métier (traçabilité, chaîne du froid, rappels) tout en conservant une base de code unique et maîtrisable.

Bien que plus structurée qu'un monolithe classique, cette architecture reste un monolithe du point de vue du déploiement.

L'application conserve donc un point de défaillance unique et ne bénéficie pas nativement des avantages de la distribution (scalabilité indépendante, isolation complète des pannes).

De plus, cette approche nécessite une discipline de conception plus importante. Sans règles claires, les frontières entre modules peuvent être progressivement contournées, ce qui réduit les bénéfices attendus.

L'architecture monolithe modulaire constitue un compromis équilibré entre simplicité et qualité architecturale. Elle est particulièrement adaptée aux projets de taille moyenne souhaitant préparer une éventuelle évolution vers une architecture distribuée, sans en subir immédiatement la complexité.

C. Architecture microservices synchrones

(REST / gRPC, base de données par service)

L'architecture microservices synchrones repose sur un découpage du système en plusieurs services indépendants, chacun responsable d'un domaine métier spécifique.

Chaque microservice est déployé de manière autonome et communique avec les autres via des appels synchrones (HTTP REST ou gRPC).

Chaque service dispose généralement de sa propre base de données afin de garantir une isolation forte.

Cette architecture offre un découplage fort entre les domaines métier, permettant des déploiements indépendants et une évolution ciblée des fonctionnalités.

Elle améliore également la disponibilité globale du système : la défaillance d'un service non critique n'entraîne pas nécessairement l'indisponibilité de l'ensemble de la plateforme.

Dans un contexte industriel à grande échelle, cette approche facilite la montée en charge, l'organisation par équipes et l'intégration de nouvelles fonctionnalités.

L'architecture microservices introduit cependant une complexité opérationnelle importante. Elle nécessite des outils et des pratiques avancées en matière de CI/CD, de monitoring, de gestion des logs et de gestion des erreurs réseau.

Dans le cadre d'un projet comme NutriChain, le risque principal est le sur-découpage prématuré. Mettre en place une architecture microservices avant d'avoir validé les besoins fonctionnels réels peut entraîner une complexité excessive sans bénéfice immédiat, notamment pour un MVP.

La gestion des transactions distribuées et la cohérence des données entre services représentent également des défis non négligeables.

L'architecture microservices synchrones est particulièrement adaptée aux systèmes complexes et à grande échelle, mais elle peut s'avérer contre-productive lorsqu'elle est adoptée trop tôt. Son intérêt est réel sur le long terme, mais son coût initial est élevé.

IX. Audit d'outils nécessaires face aux besoin du projet

Besoin	Outil / Langage
Outil de conception et pilotage du projet	
Gestion de projet	Jira / Trello
Rédaction CDC et spécificités du projet	Notion / Google Docs

Diagrammes	Draw.io / Lucidchart
Roadmap du projet	Notion / Jira
Documentation technique développeur	Markdown sur Git
Design UX/UI	
Wireframes	Figma
Design system	Figma
Prototypes dynamiques	Figma
Icônes	Material Icons
Mindmap / Visualisation des processus	Miro
Développement Frontend Web	
IDE	VSCode / Suite JetBrains / Cursor
Framework web	React / Vue / Svelte
UI	Material UI / Tailwind / Ant Design
Charts KPI	Recharts / Chart.js
Authentification	JWT / OAuth2
App admin	Web App responsive
Développement Mobile (Android et iOS)	
IDE	VSCode / Suite JetBrains / Cursor
Simulateurs	Android Studio et XCode
App mobile cross-platform	Flutter / React Native / SvelteNative
Accès caméra	SDK natif / dépendance
Scan code-barres	ZXing / ML Kit
Notifications	Firebase / OneSignal
Mode offline	Stockage local (SQLite)
Backend et API	
API Rest	Node.js / Java / Python / Rust / .NET
Gestion évènements EPCIS	API custom
Authentification	API / Keycloak

Notifications	Email / Push
Rappels produits	Workflow backend
Simulation IOT	API REST mock
Documentation API	Swagger / OpenAPI
Base de données	
Choix type BDD	Relationnelle / Non-relationnelle
Si relationnelle	PostgreSQL / MySQL
Si non-relationnelle	MongoSQL
ORM relationnel	Prisma / Sequelize
ORM non-relationnel	Mongoose
Tests et qualité	
Tests unitaires	Jest / JUnit / PyTest
Tests API	Postman
Tests E2E	Cypress
Tests mobile	Tests manuels documentés
Tests charge	k6
Qualité et couvrage	SonarQube
DevOps et CI/CD	
Versioning	Github / Gitlab
CI/CD	Github Actions
Environnements	Dev / Pré-prod / Prod
Build mobile	Pipeline automatisé
Déploiement	Container Docker sur serveur VPS
Sécurité et conformité	
Authentification	JWT / OAuth2
Rôles	Hiérarchie des rôles
Logs	Immuables
Audit trail	Qui / Quand / Quoi

RGPD	Données minimales
IOT et Chaîne du froid	
Ingestion capteurs	API REST
Données simulées	Script ou mock endpoint
Alertes	Notifications PUSH / Email

X. Benchmark des langages de développement

Développement Frontend Web				
	React	Vue	Svelte	
Performance (bundle, TTI)	Bon mais plus lourd	Bon et meilleur que React	Meilleure performance globale	
Temps de chargement	Moyen	Bon	Rapide	
Taille du bundle	~45-70 KB	~35-50 KB	~15-25 KB	
DOM Virtuel	Oui	Oui	Non	
Courbe d'apprentissage	Moyenne	Moyenne	Facile	
Ecosystème	Très large	Moyen	Petit mais croissant	
Communauté	Très grande	Grande	Petite	
Documentation	Très riche	Bonne	Bonne	
Adoption entreprise	Très élevée	Elevée	Faible	
Cas d'usage	Grandes applications complexes	Projets flexibles	Performances extrêmes et MVPs	
Prise en main personnelle des développeurs du projet	Très facile	Moyenne	Facile	

Développement mobile				
	React Native	Flutter		
Performance	Bonne	Très bonne		
Bridge natif	Oui	Non		
UI cross-plateform	Variable	Identique		
Communauté	Très grande	Grande		
Réutilisabilité du code	Elevé (++ si React en parallèle)	Elevé		
Courbe d'apprentissage	Facile (++ si React connu)	Moyenne		
Taille de l'app	Plus petite que Flutter	Plus grande que RN		
Ecosystème dépendances	Très large	Large		
UI demandante et animations	Contraintes en fonction de la demande	Très fluide		
Prise en main personnelle des développeurs du projet	Facile	Difficile		
Backend et API				
	Node.js	Java / SpringBoot	Python	Rust
Performance (RPS)	Moyenne (78k)	Bonne (243k)	Moyenne (30k)	Très bonne (320k)
Ecosystème	Très grand	Grand	Grand	Moyen
Courbe d'apprentissage	Faible	Moyenne à élevée	Moyenne	Elevée
Documentation	Très riche	Très riche	Bonne	Correcte
Communauté	Très grande	Très grande	Très grande	Plus restreinte
Temps de développement	Rapide	Plus lent que Node.js	Rapide	Lent
Facilité de tests	Bonne	Très bonne	Très bonne	Bonne

Prise en main personnelle des développeurs du projet	Facile	Moyenne	Moyenne	Difficile
---	--------	---------	---------	-----------

XI. Architecture retenue

Les exigences de ce projet impliquent une logique métier riche, fortement orientée événements, avec des règles fonctionnelles critiques (auditabilité, intégrité des données, historisation).

Dans ce contexte, le choix de l'architecture logicielle doit répondre à plusieurs objectifs :

- garantir une bonne structuration du métier,
- permettre une évolution progressive du système,
- rester réaliste et maîtrisable dans le cadre d'un projet académique réalisé par une équipe réduite.

L'architecture monolithique classique, bien que simple à mettre en œuvre, présente des limites importantes face à la diversité des domaines fonctionnels de NutriChain. Le regroupement de la traçabilité, de la gestion des alertes, de la chaîne du froid et du reporting dans un même noyau faiblement structuré entraîne un couplage fort, rendant l'évolution et la maintenance plus complexes.

À l'inverse, une architecture microservices, bien qu'adaptée à des systèmes industriels à grande échelle, introduit une complexité significative en termes d'infrastructure, de déploiement, de communication réseau et d'observabilité. Dans le cadre d'un projet académique, cette complexité représenterait un coût disproportionné par rapport à la valeur fonctionnelle réellement apportée, en particulier pour un MVP.

L'architecture monolithe modulaire permet de concilier les avantages des deux approches précédentes.

Elle repose sur une application unique, facilitant le déploiement et la mise en œuvre, tout en imposant une séparation stricte des domaines métier.

Dans NutriChain, cette séparation se traduit naturellement par des modules correspondant aux grands domaines :

- gestion du catalogue et des référentiels,
- traçabilité des lots et événements EPCIS,
- supervision de la chaîne du froid,
- gestion des alertes et rappels produits,
- reporting et indicateurs.

Chaque module est conçu comme une unité cohérente, responsable de son propre périmètre fonctionnel, ce qui améliore la lisibilité du code et réduit les dépendances croisées.

L'intégration des principes de l'architecture hexagonale renforce encore la pertinence de ce choix.

En isolant le cœur métier des préoccupations techniques (API REST, base de données, services externes, simulation IoT), NutriChain bénéficie d'un modèle où la logique fonctionnelle reste indépendante des choix technologiques.

Cette approche est particulièrement adaptée au projet pour plusieurs raisons :

- la traçabilité et les règles métier EPCIS peuvent être testées indépendamment de l'infrastructure,
- les sources de données (capteurs simulés, entrées manuelles, API externes) peuvent évoluer sans remise en cause du métier,
- l'architecture facilite la simulation de composants (IoT, alertes), ce qui correspond aux contraintes pédagogiques du projet.

L'architecture hexagonale contribue également à une meilleure testabilité, en permettant de valider les cas d'usage métier sans dépendre d'une base de données ou d'un serveur HTTP.

Un autre avantage majeur de l'architecture monolithe modulaire avec architecture hexagonale est sa capacité à préparer une évolution vers une architecture plus distribuée, sans réécriture complète du système.

Les modules métiers, déjà bien délimités et faiblement couplés, peuvent être extraits ultérieurement sous forme de microservices si le contexte l'exige (montée en charge, exigences de disponibilité accrues, organisation par équipes).

Ainsi, l'architecture retenue s'inscrit dans une logique d'évolution progressive, en cohérence avec les pratiques industrielles.

Au regard des exigences fonctionnelles de NutriChain, des contraintes opérationnelles décrites dans la synthèse détaillée et du cadre académique du projet, l'architecture monolithe modulaire s'appuyant sur les principes de l'architecture hexagonale apparaît comme le choix le plus pertinent.

Elle permet :

- une structuration claire et robuste des domaines métier,
- une maîtrise de la complexité technique,
- une forte testabilité,
- et une évolutivité raisonnée vers des architectures plus distribuées.

Ce choix offre ainsi un équilibre optimal entre rigueur architecturale, faisabilité et crédibilité industrielle, en adéquation avec les objectifs du projet NutriChain.

XII. Langages de développement et outils retenus

Les choix retenus visent à garantir un équilibre entre performance, maintenabilité et faisabilité, tout en tenant compte du contexte académique du projet et de la taille réduite de l'équipe de développement.

A. Choix du backend et de l'API : Node.js

Le backend de NutriChain est chargé de gérer :

- des APIs REST exposant les fonctionnalités métier,
- des flux événementiels liés à la traçabilité (EPCIS),
- des règles métier critiques (alertes, rappels produits),
- des interactions avec des applications web et mobiles,

- une logique de simulation pour les données IoT.

Ces besoins impliquent une API réactive, maintenable et capable de traiter un volume raisonnable de requêtes avec une latence maîtrisée.

Node.js a été retenu pour le développement de l'API pour les raisons suivantes :

- Productivité élevée : la simplicité de JavaScript et la rapidité de mise en œuvre permettent de livrer rapidement un MVP fonctionnel.
- Asynchronisme natif : le modèle event-driven de Node.js est bien adapté au traitement d'événements de traçabilité et à la gestion de flux asynchrones.
- Écosystème très mature : disponibilité de bibliothèques pour l'authentification, la validation des données, la documentation d'API et la gestion des bases de données.
- Bonne performance pour le besoin métier : les performances mesurées sont largement suffisantes au regard des volumes attendus dans NutriChain.
- Cohérence full-stack : l'utilisation de JavaScript sur l'ensemble de la stack simplifie le partage de modèles, de validations et de bonnes pratiques entre frontend, mobile et backend.

Ainsi, Node.js représente un compromis équilibré entre performance, flexibilité et rapidité de développement.

B. Choix du frontend web : Svelte

Le frontend web de NutriChain vise principalement :

- l'administration des données,
- la visualisation des indicateurs (KPI),
- la consultation des informations de traçabilité,
- l'ergonomie pour des utilisateurs professionnels.

Les priorités sont donc la performance perçue, la simplicité du code et une expérience utilisateur fluide.

Svelte a été retenu pour le frontend web pour les raisons suivantes :

- Excellentes performances : l'absence de Virtual DOM permet des temps de chargement rapides et une faible latence à l'interaction.
- Taille de bundle réduite : un bundle plus léger améliore l'expérience utilisateur, notamment sur des connexions limitées.
- Simplicité du code : la syntaxe de Svelte est plus concise et lisible, ce qui réduit la complexité du frontend.
- Adaptation aux besoins du projet : les écrans d'administration et de visualisation bénéficient directement des performances offertes par Svelte.
- Approche moderne : Svelte favorise une architecture claire et un état de l'art en matière de développement frontend.

Bien que l'écosystème soit plus restreint que celui de React, il reste suffisant pour les besoins du projet NutriChain.

C. Choix du développement mobile : React Native

Les applications mobiles NutriChain sont destinées aux opérateurs terrain, avec des contraintes spécifiques :

- utilisation sur Android et iOS,
- accès à la caméra pour le scan de QR codes et codes-barres,
- fonctionnement en mode hors ligne,
- notifications push,
- ergonomie et réactivité.

React Native a été retenu pour le développement mobile pour les raisons suivantes :

- Développement cross-platform : une base de code unique pour Android et iOS, réduisant les coûts et le temps de développement.
- Très grande communauté : abondance de plugins pour l'accès au matériel (caméra, stockage, notifications).

- Bonne performance native : suffisante pour les usages terrain (scan, navigation, synchronisation).
- Réutilisation des compétences JavaScript : cohérence avec le backend Node.js et une partie du frontend.
- Maturité industrielle : largement utilisé en production dans des applications mobiles complexes.

Ce choix permet de répondre efficacement aux contraintes terrain tout en garantissant la maintenabilité de l'application mobile.

D. Choix de la base de données principale : PostgreSQL

Le projet NutriChain doit garantir :

- l'intégrité des données,
- la traçabilité complète des événements,
- l'auditabilité et l'historisation,
- la gestion de relations complexes (produits, lots, événements, alertes).

PostgreSQL a été retenu comme base de données principale pour les raisons suivantes :

- Modèle relationnel robuste : adapté aux données structurées et fortement liées.
- Transactions ACID : essentielles pour garantir la cohérence des données de traçabilité.
- Support avancé des types de données : JSON, timestamps précis, indexation performante.
- Maturité et fiabilité : base de données largement utilisée dans des systèmes critiques.
- Adaptation à l'audit trail : possibilité de tables immuables (append-only) pour l'historique.

PostgreSQL répond ainsi parfaitement aux exigences de fiabilité et de conformité du projet.

E. Choix de l'ORM : Prisma

Prisma a été retenu pour l'accès aux données pour les raisons suivantes :

- Typage fort et sécurité : génération de types garantissant la cohérence entre le code et la base de données.
- Productivité : simplifie l'écriture des requêtes et réduit les erreurs.
- Intégration native avec Node.js : cohérence avec le backend choisi.
- Migrations maîtrisées : versionnement clair du schéma de base de données.

L'utilisation de Prisma contribue à la maintenabilité et à la robustesse du backend.

F. Usage complémentaire d'une base NoSQL : MongoDB (logging)

Pour les besoins de journalisation, de logs applicatifs et d'événements techniques, une base NoSQL de type MongoDB peut être utilisée en complément :

- Schéma flexible : adapté aux logs hétérogènes.
- Écriture rapide : optimisée pour l'ingestion de volumes élevés de données.
- Décorrélation du métier : évite de polluer la base relationnelle principale.
- Scalabilité horizontale : adaptée à la croissance des logs.

Ce choix permet de séparer les préoccupations métier et techniques tout en facilitant l'analyse et le debugging.

G. Conclusion

Les choix technologiques retenus pour NutriChain résultent d'un compromis raisonné entre performance, maintenabilité, maturité des outils et contraintes organisationnelles.

L'utilisation de Node.js, Svelte, React Native et PostgreSQL constitue une stack cohérente, moderne et adaptée aux exigences fonctionnelles du projet, tout en restant réaliste dans un cadre académique.

Cette stack permet également une évolution progressive du système, tant sur le plan fonctionnel que technique.